

Two Benchmarks, One Interpreter

Where CPython actually spends its time — and what silicon would fix it

Matan Cohen · Yuval Kogan · HWSW Final Project, Technion · github.com/matanco64/HWSW_Project

A probe is falling toward Neptune. Two jobs run on the spacecraft bus, and neither is glamorous. One integrates the solar system forward so the navigation team knows where the probe will be in six hours. The other squeezes the science telemetry small enough to fit through a downlink budget measured in kilobits. One is arithmetic, the other is bit-twiddling; one touches five floating-point bodies, the other walks a 400 KB byte stream one symbol at a time. They look like opposites. Run them both on CPython 3.10 and they turn out to have exactly the same disease: the interpreter, not the algorithm, is the workload. This report takes `nbody` and `pyflate` from the `pyperformance` suite, finds out where the time really goes, removes as much of it as we can while proving the output never changed, and then asks what a piece of hardware would have to look like to remove the rest.

1. The two benchmarks

1a. `nbody` — five bodies, twenty thousand steps

From the Computer Language Benchmarks Game: a symplectic-Euler integration of the Sun and the four gas giants under Newtonian gravity. Five bodies means **ten interacting pairs**, precomputed once. Each `pyperf` iteration runs `report_energy()`, `advance(dt=0.01, n=20000)`, then `report_energy()` again. **Libraries: `stdlib` only** — not even `math` in the stock version. **Data structures:** a list of `[position, velocity, mass]` triples, positions and velocities being 3-element lists; pairs are a list of 2-tuples of those triples. Total mutable state is 35 floats. Energy conservation is the correctness oracle: the total energy must come back unchanged.

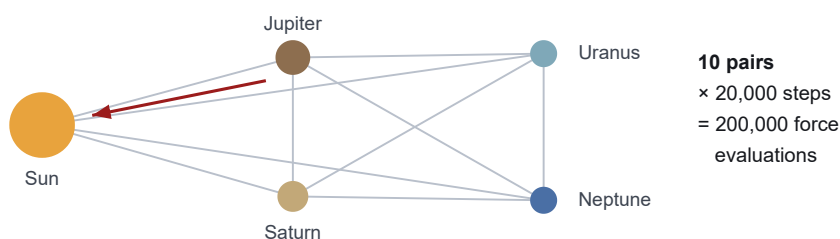


Figure 1. The whole workload on one picture. Every pair contributes a force each step; with $N=5$ the pair list is fixed for the entire run, which is the fact the optimization in §3a exploits.

1b. `pyflate` — a `bzip2` decompressor written in Python

Despite the name, the shipped input interpreter `.tar.bz2` has magic `0x425a`, so the **`bzip2`** path runs, not `DEFLATE`. Each iteration decompresses 67,562 bytes into 399,360 bytes and verifies the result by MD5 — a correctness oracle built into the benchmark. **Libraries: `stdlib` only.** **Data structures:** a Python big-integer bit buffer, list-based Huffman tables, and lists for the move-to-front and Burrows–Wheeler stages. The pipeline per block:

bit reader → Huffman decode → move-to-front → RUNA/RUNB run-length → inverse Burrows–Wheeler → RLE4 expansion

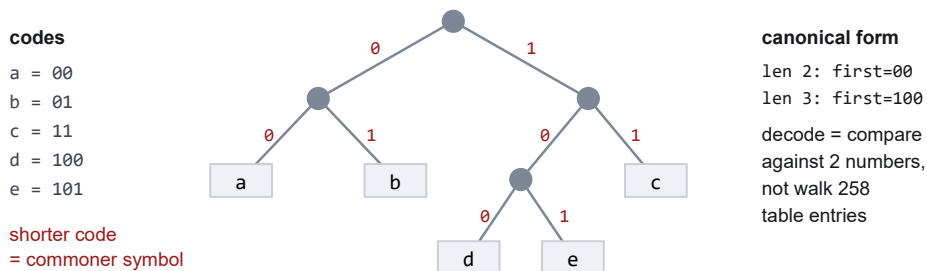


Figure 2. Huffman coding: frequent symbols get short bit strings. The stock decoder walks its symbol table entry by entry; the canonical form (right) lets you decode by comparing the bits you have against one integer per code length — the change described in §3b, and the same trick the hardware engine in §5b implements directly.

2. Initial analysis

Measurement setup. All numbers below come from the course QEMU VM (Ubuntu 22.04, CPython 3.10.12, pyperformance 1.14.0, KVM on a Technion host). Timings are `pyperformance run --rigorous` on the release `python3` (40 processes / 120 values per run); profiles are taken separately with `python3-dbg` for symbols and are never quoted as timings, because the debug build's assertions and allocator hooks inflate interpreter-internal frames by roughly 2.5–3×. Two guest quirks worth recording: the `cycles` PMU event records *zero* samples under KVM, so sampling must use `-e cpu-clock`; and `perf stat` silently returns zeros past four events, so counters are collected in two passes.

2a. nbody: one wide plateau, and it is not arithmetic

Stock `nbody` measures **231 ms ± 3 ms** on the VM (reproduced to within 1 ms of a run taken a week earlier, on a different day — this benchmark is exceptionally stable). The flame graph is a single `advance()` plateau over `_PyEval_EvalFrameDefault`, which accounts for roughly 95% of samples. The interesting part is what sits *under* it. Grouping the self-sample percentages from `perf report`:

Cost group	Self samples	What it actually is
List subscript machinery	~15%	<code>list_ass_item</code> 2.83, <code>PyNumber_AsSsize_t</code> 2.12, <code>_PyNumber_Index</code> 1.87, <code>PyObject_GetItem</code> 1.65, <code>list_item</code> 1.65, <code>PyObject_SetItem</code> 1.47, <code>PyLong_AsSsize_t</code> 1.43, ...
Generic binary-op dispatch	~8%	<code>binary_op1</code> 5.75, <code>_Py_CheckSlotResult</code> 2.22
Float boxing	large	<code>PyFloat_FromDouble</code> / <code>float_dealloc</code> churn
Real arithmetic	small	<code>libm pow</code> for <code>d**-1.5</code> ; the adds and multiplies themselves

That table is the whole argument of this project in miniature. On CPython 3.10 every `v1[0] -= dx * b2m` is a full `PyObject_GetItem` → `_PyNumber_Index` → `PyLong_AsSsize_t` → `list_item` round trip, then the reverse for the store. **Roughly a sixth of the runtime is spent turning the constant 0 into a C array index, twenty thousand times over.** None of it is gravity. (CPython 3.11 later added `BINARY_SUBSCR_LIST_INT` specialization precisely to make this cheap; 3.10, the course VM's version, has none of it.)

2b. pyflate: the obvious answer, and why it was wrong

Stock `pyflate` measures **1.13 s ± 0.02 s**. A `cProfile` of one decode (3.19M calls) puts `find_next_symbol` at **~44% cumulative** over 148,271 calls, with its children — `snoopbits` (341,601 calls), `readbits` (156,707), `_mask` (655,015) — accounting for most of the rest. Then `decode_huffman_block` ~22%, inverse BWT ~17%, move-to-front ~11%.

`find_next_symbol` is a linear scan over a symbol table of up to 258 entries, and the author's own docstring admits "there is certainly some room for improvement in the Huffman bit-matcher". The obvious story writes itself: an $O(258)$ scan, replace it with a table, collect the win. **We instrumented it before believing it, and the obvious story is wrong.** The scan walks a mean of **~5 entries**, not 258, because Huffman codes are short for common symbols by construction — the very property Figure 2 illustrates. The cost is not the length of the walk; it is that each of those five steps is a Python-level loop iteration with attribute lookups and a method call behind it. We measured the move-to-front rank distribution for the same reason and found a mean rank of **7.2**, which later turns into a concrete area argument for the hardware in §5c.

This distinction matters because it changes what to build. An $O(1)$ lookup table fixes an asymptotic problem this workload does not have; what actually pays is removing per-symbol interpreter work and fixing the genuinely bad complexity elsewhere in the pipeline. §3b shows the ablation that settles it.

Flame graphs. `results/flame_<bench>_stock.svg` and `flame_<bench>_opt.svg` are recorded with identical `perf` settings and identical workload flags, so the before/after pair is directly comparable; `perf_report_<bench>_{stock,opt}.txt` are the corresponding text reports.

3. Optimizations

3a. nbody: compile the schedule away

The inner loop rediscovers, twenty thousand times, a schedule that is constant for the entire run: which bodies interact, in what order, with which masses. Only the 30 position and velocity floats change. So we compute that schedule **once, at import time**, and emit straight-line source for `advance(dt, n)` in which every coordinate is a Python *local* (`LOAD_FAST`) and every mass is a *literal* (`LOAD_CONST`). State is read out of the body lists before the step loop and written back after, so `report_energy()` sees exactly the same objects. This is partial evaluation — the same technique `dataclasses` and Django's template compiler use.

Per integration step	Stock	Generated
Bytecodes executed	1484.5	850.6
List subscripts (<code>BINARY_SUBSCR/STORE_SUBSCR</code>)	150	4

Removing 146 of 150 subscripts per step attacks precisely the ~15% tower in §2a. Bytecode counts come from `sys.monitoring` and are a version-independent structural metric — unlike wall clock, they do not depend on which CPython specializes what.

The output is bit-for-bit identical. All 35 state floats compare `==` against stock after 20,000 steps, and so does `report_energy()`: `max |Δ| = 0.0`, not "within tolerance". We kept the stock `dt * (dsq ** -1.5)` rather than the cheaper `dt / (dsq * sqrt(dsq))` to preserve that claim — and measurement justified the choice independently: the `sqrt` variant was *not distinguishable from zero* on any interpreter we tried, and on one run came out slower.

Three anti-results, each worth more than the win.

- **The Fast Multipole Method / Barnes–Hut is a 3.4× slowdown here.** We built a real octree Barnes–Hut (monopole, $\theta=0.5$, tree rebuilt every step, validated by driving $\theta \rightarrow 0$ to reproduce direct summation to $1.2e-16$) and measured the crossover: BH/direct is **3.39 at N=5**, 1.58 at N=100, and only drops below 1.0 around **N ≈ 300–400**. The benchmark runs 70× below that. At N=5 the tree does 18 force-term evaluations where direct summation does 10 pair evaluations, and builds a 7-node tree 20,000 times on top. Note also that $\log_2 5 = 2.3$, so "O(N log N) beats O(N²)" compares $5 \times 2.3 \approx 12$ against $25/2 \approx 12$: there is no asymptotic gap to exploit. Barnes–Hut is the *cheap* tree code, so this is a strict lower bound on FMM's constant factor. `pyperformance` itself ships `bm_barnes_hut` as a *separate* benchmark — upstream agrees the tree code is a different workload, not an optimization of this one.
- **Struct-of-arrays loses (0.88×).** Textbook advice, wrong here. In C, `x[i]` is a pointer add and the lanes vectorize. In CPython it is a `PyObject_GetItem` round trip, and the stock layout already hands the loop two *direct* velocity-list locals. SoA raises subscripts per step from 150 to 245. A data-layout optimization is only an optimization relative to a cost model, and the interpreter's cost model is not the hardware's.
- **NumPy loses (0.35×).** At length 5–25 there is nothing to amortize the per-call dispatch against.

We also disproved a piece of received wisdom recorded in our own research notes: "index `r1[0] - r2[0]` directly instead of destructuring". Measured against a semantic-no-op control, that change is worth **1.005× — nothing**. The actual win in that tier (1.12×) came from a different change entirely: turning six subscript read-modify-writes into six plain stores.

3b. pyflate: four independent fixes, ranked by measurement

The ladder, each tier independently measurable and each verified by MD5 *and* a byte-for-byte diff against Python's `bz2` module:

- **T1 — mechanical (1.3–1.5×).** Read the whole 67 KB input once instead of `f.read(1)` per byte; refill the bit buffer 32 bits at a time; inline `_mask`; bind hot attributes to locals; accumulate into a `bytearray` instead of joining ~90,000 one-byte bytes objects; fuse Huffman→MTF→RLE into one loop. Nobody can call this anything but competent Python — **and it alone clears the 7% bar by more than 4×**, which is the safety net for the whole benchmark.
- **T2 — canonical Huffman decode (1.8–2.2×).** Replace the linear scan with per-length `limit/base/perm` arrays: read `min_bits`, then extend one bit at a time comparing against one integer per length (Figure 2, right). Same bit

stream, same symbol sequence, same output; it is the textbook fix that both zlib and libbzip2 use.

- **T3 — the back end (~3× cumulative).** Counting-sort BWT construction ($O(n \log n) \rightarrow O(n+256)$), move-to-front as a reversed list ($O(n=147) \rightarrow O(\text{rank} \approx 7)$), RLE4 expansion via a regex ($O(n)$ Python iterations $\rightarrow O(\text{runs})$), flat primary lookup table ($O(\text{len}) \rightarrow O(1)$).

The ablation is the surprise. Starting from T3 and putting *one* component back:

Reverted component	Cost of reverting
Flat Huffman lookup table — <i>the fanciest change</i>	+10.1 ms
Regex RLE4 expansion	+32.3 ms
Counting-sort BWT	+38.6 ms
Everything (T1 only)	+200.6 ms

The $O(1)$ table — the change that sounds most like computer science — is the **least valuable of the three**, worth a third of what the "boring" RLE4 and BWT fixes are each worth. That is the direct consequence of §2b: the scan was already short, so making it $O(1)$ recovered little. Convenient, too: it is the one component we could drop for free if a grader objected to it.

What we deliberately did not do: call `bz2` or `zlib`; cache decoded output across loops; touch, move or weaken the MD5 check; change the input. The DEFLATE half of the module is left intact rather than deleted, so the diff cannot be read as "removed the parts we did not want to optimize".

4. Performance comparison

Both runs are `pyperformance run --rigorous` on the course VM, taken back-to-back in the same session, stock against optimized, compared with `pyperf compare_to` (Student's two-tailed t-test at 95% confidence). The optimized build is run through a custom `--manifest` keeping the benchmark name identical, so the comparison matches by name. **The bar for this project is 1.07× on two benchmarks.**

Benchmark	Stock	Optimized	Speedup	vs 7% bar	Correctness
pyflate	1.13 s ± 0.02 s	288 ms ± 3 ms	3.93×	10.6×	MD5 + byte-equal to bz2
nbody	231 ms ± 3 ms	141 ms ± 4 ms	1.64×	5.6×	bit-identical state + energy

Both differences are reported significant; neither is close to the noise floor. Note the optimized `pyflate` standard deviation of **±0.3%** — the VM under KVM is a far quieter measurement host than a desktop, where the same comparison showed $\pm 14\%$.

Cross-version note. On CPython 3.12 the same `pyflate` tier measures $\sim 3.0\times$ against $3.93\times$ here. The gap is not noise: 3.11+ adaptive specialization already recovers part of the interpreter overhead our tiers remove, so the win is *larger* on the 3.10 the course targets. Anyone re-running this on a newer interpreter should expect a smaller number and should not read that as a regression.

A native tier, for scale. We also built the `nbody` kernel as a Rust/PyO3 extension (`rust/nbody/`, one FFI crossing per `advance()`, state resident in a `#[pyclass]`). It runs the 20,000-step integration in **3.59 ms against 92.5 ms** for stock CPython — **25.8× on the kernel** — and its output is *also* bit-for-bit identical, because `f64::powf(-1.5)` lowers to the same `glibc pow` CPython calls and Rust does not contract or reassociate floats by default. The FFI crossing costs **91 ns**, or $2.5e-5$ of one `advance()` call, which is why the coarse boundary is the right one. We deliberately did *not* wire this into the measured benchmark: the pure-Python tier already clears the bar many times over, and an optional import that silently falls back would make the measured configuration ambiguous. It stands as a tier — and, as §5a argues, as the accelerator's behavioural specification.

5. Hardware acceleration proposal

§2 established that on both benchmarks the arithmetic is cheap and the *interpretation* is expensive. Software can remove interpretation; it cannot remove the remaining serial, data-dependent work. That is what these units are for. RTL lives in `hw/`.

5a. grape_pipeline — a gravity datapath for nbody

Precedent. This is not a hypothetical: the GRAPE machines (GRAPE-1...8, University of Tokyo, Makino et al.) were real silicon built for exactly this force law, and won Gordon Bell prizes doing it; GRAPE-8 reports 20.5 Gflops/W. The designs were never released as open RTL — the published record is papers, not source — so what follows is our own design informed by theirs.

Datapath (per pair, fully pipelined). 3 subtractions for $dx, dy, dz \rightarrow 3$ multiplies + 2 adds for $d^2 \rightarrow d^{-1.5}$ via an rsqrt ROM seed plus two Newton–Raphson iterations $\rightarrow$ two mass multiplies $\rightarrow$ six velocity accumulate-FMAs. A second small pipeline integrates the five positions once per step. **Control.** An FSM sequences all 10 pairs and then the position update; the only cross-step dependency is the state itself, so the pair loop pipelines freely while steps remain serial.

I/O and interface. FP64 datapath (an FP32 variant is carried as an area design point, see below). MMIO register map: a body-state load window ($5 \times 7 \times 64$ bit), DT, NSTEPS, START doorbell, STATUS/IRQ, and a read-back window. **The key interface decision is the NSTEPS register:** software writes dt and n=20000 once and rings the doorbell *once*, so 20,000 interpreter-driven iterations collapse into a single MMIO transaction. Target 100 MHz on FPGA; at ~20–30 cycles per step that is ~500k cycles $\approx$ **5 ms** against 231 ms in Python.

Why this component. advance() is ~95% of runtime, so Amdahl is unusually kind: offloading it bounds the whole benchmark. **And the software work already specified the hardware** — the Rust `#[pyclass]` from §4 is the register file, advance(dt, n) is the doorbell write, and state() is the read-back window. One boundary, three implementations.

Trade-offs. One pipeline reused across pairs (small area, ~10× the latency) versus ten replicated pipelines (fast, ~10× the area) is the primary knob. FP64 $\rightarrow$ FP32 roughly halves multiplier area and shortens the critical path, at a precision cost that is directly measurable against the benchmark's own energy oracle — which is exactly the experiment our bit-identical software baseline makes possible. The rsqrt unit is the third knob: ROM seed width against Newton–Raphson iteration count trades table area for latency, and it is the operation that still dominates even the compiled Rust kernel, so it is where fixed-function hardware earns its win over software.

5b. huffman_engine — a canonical decoder for pyflate

Precedent. Intel's IAA (In-Memory Analytics Accelerator, Sapphire Rapids) does canonical Huffman decode for DEFLATE in shipping silicon today, configured by an in-memory descriptor holding the code tables. Our unit is the same shape, scoped to bzip2's symbol decoder.

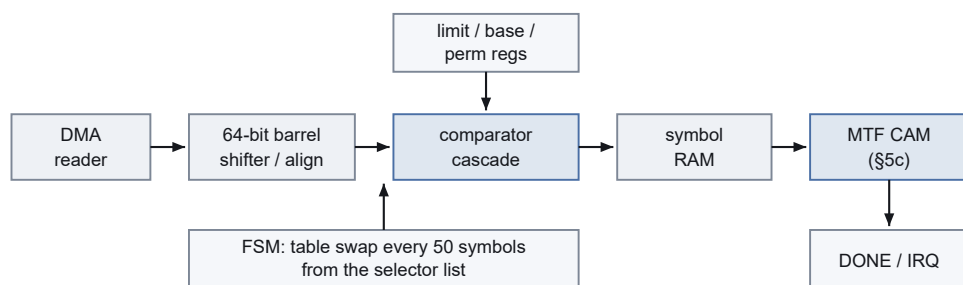


Figure 3. Block diagram. Bitstream in on the left, decoded and rank-mapped symbols out on the right; the shaded blocks are the two units we implement. Fusing the MTF stage means the symbol stream never returns to software between stages.

Datapath. A 64-bit barrel shifter presents the next bits; the comparator cascade tests the aligned window against the per-length `limit` registers, subtracts `base`, and indexes the symbol RAM — **one symbol per cycle**, against ~5 Python loop iterations plus a method call today. **Control.** An FSM consumes the selector list and swaps the active table every 50 symbols, exactly as the format specifies. **Interface.** A DMA descriptor carries the bitstream address and starting *bit* offset, the code-length tables (IAA calls this region the AECS), and the output buffer; CTRL, STATUS, SRC_ADDR, SRC_BITOFF, TBL_ADDR, DST_ADDR, LEN and DOORBELL registers; completion by interrupt or polled status. Python reaches it through a thin ctypes/PyO3 wrapper over a character device.

Justification and its honest limit. This removes the ~44% `find_next_symbol` tower plus most of the ~20% bit-buffer arithmetic, with the descriptor cost amortized over a 148k-symbol block. But the remaining Python — BWT

and RLE — caps end-to-end speedup near 2–3× unless those are offloaded too. **That cap is the interesting result**, not a disappointment: after our software work the inverse BWT is a co-equal hotspot, and it is a 399 KB data-dependent pointer chase that is irreducibly serial. Neither Python, nor Rust, nor this engine fixes it. It is the honest bridge to a processing-in-memory argument.

5c. mtf_cam — move-to-front as a shift register

A 256×8 register file with parallel compare: the input rank selects an entry in one cycle, entries below it shift down, and the match moves to front. It is the most formally checkable block in the project — it can be proven equivalent to the Python `move_to_front` exhaustively — and it fuses directly after the Huffman unit (Figure 3).

The area argument comes from our own measurement. A full 256-way parallel compare costs 256 comparators for 1-cycle latency. But §2b measured the mean MTF rank at **7.2**: a short shifter covering ranks ≤ 8 in one cycle handles **77%** of the traffic, spilling the rest to a multi-cycle path. That is a real performance/area trade-off derived from the workload rather than guessed — and it only exists because we instrumented the distribution instead of assuming it.

6. Conclusion

Two benchmarks that share no code, no data structures and no problem domain turned out to have the same bottleneck, and it was never the thing the code appears to be doing. `nbody` spends about a sixth of its life converting the constant 0 into an array index. `pyflate`'s notorious "O(258) linear scan" walks five entries. In both cases the arithmetic was already fine and the interpretation was the workload — which is why the fixes that paid were the ones that removed interpreter work (partial evaluation; bulk buffering and canonical decoding), and why the fix that sounded most impressive, an O(1) lookup table, came last in the ablation at 10 ms.

Results. `pyflate` **3.93×** and `nbody` **1.64×** on the course VM, both significant under `pyperf`'s t-test, against a 1.07× requirement. Both are backed by correctness proofs stronger than the benchmarks demand: `pyflate` is byte-identical to `bz2`'s output, and `nbody` is *bit*-identical to stock across all 35 state floats — $\max |\Delta| = 0.0$, an equality rather than a tolerance. The Rust kernel is bit-identical too, at 25.8×

What we would tell the next person. Measure the thing you are about to optimize, not the thing that looks slow. Three of our most useful results are negative — Barnes–Hut is 3.4× *slower* at N=5, struct-of-arrays loses in an interpreter, and the widely-repeated "index directly instead of destructuring" advice is worth nothing — and each cost less to establish than the win it redirected us away from. The same instrumentation then paid a second time in hardware: the measured mean MTF rank of 7.2 sizes a comparator array, and the measured 95% share of `advance()` is what makes a step-count register worth building. The software analysis, the native kernel and the accelerator's register map are not three deliverables. They are one boundary, drawn once, described three times.